

CLEARRECORD AI

Product Requirement Document (PRD)

PLATFORM

Website

FRONTEND STACK

Other (Next.js)

BACKEND & DATABASE

Firebase

VISUAL STYLE & THEME

Playful (Spotify Dark)

FONT FAMILY

Lexend

GENERATED ON

10/2/2026

Product Overview

Problem Statement

Most employers include social media screening as part of their pre-employment background checks, but candidates are rarely aware that years of old, often out-of-context content (such as angry tweets posted in frustration, vulgar comments made in casual conversations, or old party photos) can flag them as high-risk hires. This is especially impactful for recent college graduates with 5+ years of social media activity, working professionals switching jobs with 10+ years of digital footprints, and career changers moving to regulated industries where social media screening is standard practice. Currently, there is no accessible, purpose-built tool that lets users audit their own social media presence from an employer's perspective before applying for roles, leading to unexpected disqualifications and lost job opportunities.

Solution Summary

ClearRecord AI is a web-based pre-employment privacy tool that empowers job seekers to proactively audit their social media presence for hiring risks. Users upload their official social media data archives from major platforms, and the tool's AI scans all content against 13 common employer screening risk categories (workplace hostility, aggressive debates, profanity, substance references, etc.). Each flagged item is categorized as High, Medium, or Safe risk, and users can review, delete, or untag risky content directly in the tool. The tool generates a quantifiable Digital Hygiene Score (0-100) and a downloadable PDF report that users can use to demonstrate proactive digital hygiene management to employers.

Target Audience

1. **Recent College Graduate**

- Role: Final-year student or 0-2 years into their first full-time role
- Primary Goals: Clean up immature or unprofessional social media content before applying for jobs
- Key Needs: Simple, low-effort audit process, clear explanations of what content is flagged and why
- Pain Points: Unaware of how old teenage/young adult posts will be interpreted by employers, no easy way to review years of scattered social media content
- Product Support: Guided upload flow, plain-language risk explanations, one-click actions to remove risky content

2. **Working Professional (Job Switcher)**

- Role: 25-35 year old with 5+ years of professional experience looking to change roles
- Primary Goals: Identify and remove controversial or unprofessional old content that could disqualify them from new opportunities

- Key Needs: Support for multiple social platforms, ability to review content in context before deleting, secure handling of personal data
- Pain Points: Forgot about old controversial posts, worried about past content being taken out of context, no way to see their digital footprint from an employer's perspective
- Product Support: Multi-platform archive support, context-aware risk categorization, end-to-end encryption for uploaded data

3. ****Career Changer (Regulated Industry)****

- Role: 30-40 year old moving to a highly regulated industry (finance, healthcare, education, government)
- Primary Goals: Ensure their entire digital footprint complies with the stricter screening standards of their new industry
- Key Needs: Granular filtering of risk categories, detailed score breakdowns, employer-facing report options
- Pain Points: Old content that is harmless in their current industry may be flagged as high-risk in regulated fields, no way to tailor their audit to industry-specific standards
- Product Support: 13 granular risk categories, category-specific filtering, redacted employer-facing report options

Core Value Proposition

- Proactively identify hidden hiring risks in your social media history before employers screen you, eliminating unexpected disqualifications
- Get an objective, AI-powered audit categorized by 13 common employer screening risk factors, with clear context for every flagged item
- Take direct action to clean up risky content from within the tool, no need to manually scroll through years of scattered posts across multiple platforms
- Quantify your digital hygiene with a 0-100 score and shareable PDF report to demonstrate proactive privacy management to prospective employers
- Avoid the stress and uncertainty of unknown social media risks by getting full visibility into your digital footprint before applying for roles

Success Goals

1. 80% of users complete the full audit workflow (upload ' scan ' review ' score generation) within 10 minutes
2. 90% of first-time users identify at least 1 previously unknown risky content item during their initial audit
3. < 2 second average page load time for all core screens on 4G mobile connections
4. 99.5% service availability for core scanning and reporting features

- 5. 75% of users who complete an audit download the generated PDF report
- 6. < 1% error rate for the end-to-end social media content scanning workflow

Features & Requirements

MoSCoW Feature List

Must-Have

- **Social Media Archive Upload**
- User triggers upload via drag-and-drop zone or file picker button on the upload screen
- System accepts standard social media archive formats (JSON, CSV, ZIP) from Facebook, Instagram, Twitter/X, LinkedIn, and TikTok
- System validates file size (max 2GB per upload) and format, displays a clear error message for invalid files with a retry option
- System encrypts uploaded archives at rest in Firebase Storage
- **AI-Powered Risk Scanning**
- User initiates scan immediately after successful file upload
- System scans all uploaded content against 13 predefined hiring-risk categories: workplace hostility, aggressive debates, profanity, substance references, discriminatory language, unprofessional behavior, confidential information sharing, fake news dissemination, extremist content, inappropriate media, privacy violations, dishonest behavior, and violent threats
- System categorizes each flagged item as High, Medium, or Safe risk based on severity and contextual analysis
- System displays real-time scan progress with an animated progress bar and estimated time remaining
- **Risk Review Feed**
- User accesses the review feed automatically after scan completion
- System displays all flagged items sorted by risk level (High ' Medium ' Safe) with full context (post text, media preview, original platform, date posted)
- User can take action on each item: mark as reviewed, delete (if linked to a connected account), or untag/flag for manual follow-up
- System updates the Digital Hygiene Score in real-time as the user takes action on items
- **Digital Hygiene Score & Report Generation**
- System calculates a 0-100 Digital Hygiene Score based on the number and severity of flagged items, user actions taken, and total content volume

- User can view a breakdown of their score by individual risk category
- User can download a shareable PDF report of their full audit results, score, and recommended actions

Should-Have

- ****User Account Management****
- User can create an account using email/password or Google Sign-In
- Authenticated users can log in to access past audits and saved reports
- User can permanently delete their account and all associated data
- ****Multi-Platform Archive Support****
- System correctly parses platform-specific archive formats for 5+ major social platforms (Facebook, Instagram, Twitter/X, LinkedIn, TikTok) to extract posts, comments, media, and tags
- ****Scan History Dashboard****
- Authenticated users can view a list of all past audits, including upload date, final score, and number of flagged items
- User can re-run a scan on a previously uploaded archive to get updated risk assessments
- ****Risk Category Filtering****
- User can filter the review feed by risk category (e.g., only show profanity-related items)
- User can filter the review feed by risk level (High, Medium, Safe)

Could-Have

- ****Bulk Action Support****
- User can select multiple items in the review feed and apply bulk actions (mark as reviewed, delete, untag) to reduce manual work
- ****Personalized Recommendations****
- System generates actionable, personalized recommendations for improving the user's Digital Hygiene Score based on their specific flagged content
- ****Employer-Facing Redacted Report****
- User can generate a redacted version of the PDF report that only includes the overall score and high-level risk category breakdown, with no specific post details, for sharing with employers
- ****Scheduled Content Deletion****

- User can schedule deletion of risky content for a future date if they want to review it further before removing

Data to Store

Entity Name	Field Name	Data Type	Required	Key Status	Purpose
User	user_id	UUID	Yes	Primary Key	Unique user identifier
User	email	VARCHAR(255)	Yes	Unique	User login email
User	full_name	VARCHAR(255)	Yes	No	User display name
User	auth_provider	ENUM('email', 'google')	Yes	No	Authentication provider used
User	password_hash	VARCHAR(255)	No	No	Hashed password for email auth users
User	created_at	TIMESTAMP	Yes	No	Account creation timestamp
Audit	audit_id	UUID	Yes	Primary Key	Unique audit identifier
Audit	user_id	UUID	Yes	Foreign Key (User.user_id)	Owner of the audit
Audit	upload_date	TIMESTAMP	Yes	No	Date archive was uploaded
Audit	scan_status	ENUM('pending', 'processing', 'completed', 'failed')	Yes	No	Current scan status
Audit	digital_hygiene_score	INT	No	No	Final 0-100 Digital Hygiene Score
Audit	total_items_scanned	INT	No	No	Total content items scanned
Audit	total_flagged_items	INT	No	No	Total flagged items found
Audit	archive_file_path	VARCHAR(1024)	Yes	No	Path to uploaded archive in Firebase Storage

FlaggedItem	flagged_item_id	UUID	Yes	Primary Key	Unique flagged item identifier
FlaggedItem	audit_id	UUID	Yes	Foreign Key (Audit.audit_id)	Audit the item belongs to
FlaggedItem	content_text	TEXT	No	No	Text of the flagged post/comment
FlaggedItem	media_url	VARCHAR(1024)	No	No	URL to associated media in Firebase Storage
FlaggedItem	original_platform	ENUM('facebook', 'instagram', 'twitter', 'linkedin', 'tiktok')	Yes	No	Platform the content was posted on
FlaggedItem	post_date	TIMESTAMP	Yes	No	Original post date
FlaggedItem	risk_category	ENUM('work-place_hostility', 'aggressive_debates', 'profanity', 'substance_references', 'discriminatory_language', 'unprofessional_behavior', 'confidential_info_sharing', 'fake_news', 'extremist_content', 'inappropriate_media', 'privacy_violations', 'dishonest_behavior', 'violent_threats')	Yes	No	Assigned risk category
FlaggedItem	risk_level	ENUM('high', 'medium', 'safe')	Yes	No	Risk severity level
FlaggedItem	user_action	ENUM('pending', 'reviewed', 'deleted', 'untagged')	Yes	No	Action taken by the user

Report	report_id	UUID	Yes	Primary Key	Unique report identifier
Report	audit_id	UUID	Yes	Foreign Key (Audit.audit_id)	Audit the report is for
Report	report_type	ENUM('full', 'employer_redacted')	Yes	No	Type of report generated
Report	pdf_url	VARCHAR(1024)	Yes	No	URL to generated PDF in Firebase Storage
Report	generated_at	TIMESTAMP	Yes	No	Report generation timestamp
Report	expires_at	TIMESTAMP	Yes	No	Date report is deleted (30 days post-generation)

UI/UX Requirements

Color Token Table

Token	Hex Value	Role
Primary	#1DB954	Core interactive elements, CTAs, score highlights, and active states
Secondary	#191414	High-emphasis static surfaces, dark mode base background for header/footer
Accent	#1ED760	Secondary interactive elements, hover states, success indicators, and score highlights
Background	#121212	Full page base background, modal backdrops
Surface	#181818	Elevated surfaces, cards, form containers, and input backgrounds
Text	#FFFFFF	All primary text content, headings, body copy, labels, and button text

Font Family

Font Family: Lexend (exact as specified, no substitution)

Design Guidance

- Spacing System**: 8px base grid, all spacing values are multiples of 8 (8, 16, 24, 32, 48, 64px) for consistent layout across all screens
- Border Radius**: 8px for cards, buttons, form inputs, and tags; 12px for modals and large containers; 4px for small badges and indicators
- Elevation and Shadows**: No heavy drop shadows; use subtle 1px solid #FFFFFF (10% opacity) borders for elevated surfaces to align with the playful Spotify dark style
- Button States**:
 - Default: Primary background (#1DB954), white text, 8px border radius, 16px horizontal padding, 12px vertical padding
 - Hover: Accent background (#1ED760), 1.02 scale transition, 200ms ease

- Pressed/Active: Darker green (#169c46), 0.98 scale transition
- Focus: 2px solid #1ED760 outline, 4px offset from element
- Disabled: 40% opacity, no hover/active effects, cursor not-allowed
- Loading: White spinner centered in button, "Processing..." text at 60% opacity
- ****Form Controls****: Inputs have Surface background (#181818), 1px solid #FFFFFF (20% opacity) border, 8px border radius, 12px padding. Validation error state uses 1px solid #FF4444 border with error text below input. Validation success state uses 1px solid #1DB954 border with a success checkmark icon.
- ****Responsive Behavior****: Mobile-first design with breakpoints at 768px (tablet) and 1024px (desktop). Mobile uses single-column layout, full-width cards/buttons, and bottom navigation for core actions. Tablet uses 2-column grid for dashboard and review feed. Desktop uses 3-column layout with a max-width 1200px centered container.
- ****Theme Support****: System theme only (no manual toggle). The UI automatically detects the user's OS/browser theme preference and applies the corresponding color tokens. The provided dark palette is the default; light mode uses the following recommended derived tokens: Background: #FFFFFF, Surface: #F5F5F5, Text: #121212 (all other tokens remain unchanged).
- ****Playful Style Implementation****: Use smooth micro-interactions (scale transitions, animated progress bars, bounce animations for score updates), friendly rounded iconography, bright accent color for positive feedback, approachable copy tone, and subtle screen transitions to create a welcoming, non-intimidating experience.
- ****Accessibility****:
 - WCAG 2.1 AA compliant: All text has minimum 4.5:1 contrast ratio (all provided colors meet this requirement)
 - Minimum 48x48px touch targets for all interactive elements
 - Full keyboard navigation support with visible focus states for all focusable elements
 - Semantic HTML structure for all screens, proper ARIA labels for icons and non-text elements
 - Font scaling supports up to 200% zoom without layout breakage
 - Screen reader announcements for scan progress, score updates, and action confirmations

Core Screens

1. ****Upload Screen****
 - Purpose: Allow users to upload their social media data archive for scanning

- Wireframe structure (top to bottom): Header with logo and navigation ' Hero section with headline "Check Your Digital Hygiene Before Your Next Job Application" ' Drag-and-drop upload zone with file picker button ' Supported platforms list (Facebook, Instagram, Twitter/X, LinkedIn, TikTok) ' Hidden error message area ' Footer with privacy policy link
- Key UI components: Drag-and-drop zone, file picker button, platform icons, error toast, privacy badge
- Primary user action: Upload a valid social media archive file
- States: Empty (default, no file uploaded), Loading (file uploading, progress bar), Error (invalid file type/size, error message with retry button), Success (file uploaded, auto-navigate to scan screen)

2. **Scan Progress Screen**

- Purpose: Show real-time progress of the AI risk scanning process
- Wireframe structure (top to bottom): Header with logo and navigation ' Progress header with "Scanning Your Content" headline ' Animated progress bar with percentage complete ' Estimated time remaining text ' List of completed scan steps (Upload validated, Parsing content, Analyzing risk categories, Generating score) with checkmarks for completed steps ' Cancel scan button
- Key UI components: Progress bar, step list, cancel button, loading spinner
- Primary user action: Wait for scan to complete or cancel scan
- States: Processing (scan in progress, steps updating), Completed (auto-navigate to review feed), Error (scan failed, error message with retry button), Cancelled (return to upload screen)

3. **Risk Review Feed Screen**

- Purpose: Allow users to review flagged content items and take action
- Wireframe structure (top to bottom): Header with logo and navigation ' Score summary card (current Digital Hygiene Score, number of flagged items) ' Filter bar (filter by risk level, filter by risk category) ' List of flagged item cards (sorted by risk level) ' Bulk action bar (appears when items are selected) ' Load more button
- Key UI components: Score card, filter dropdowns, flagged item cards, action buttons (mark reviewed, delete, untag), bulk action bar, load more button
- Primary user action: Review flagged items and take action to clean up risky content
- States: Loading (loading initial feed), Empty (no flagged items, "Great job! No risky content found" message), Error (failed to load feed, retry button), Partial loaded (loading more items)

4. **Score & Report Screen**

- Purpose: Display the final Digital Hygiene Score and allow report download
-

Wireframe structure (top to bottom): Header with logo and navigation ' Large score display (0-100 number, color-coded: red <50, yellow 50-79, green 80-100) ' Score breakdown by risk category (bar chart) ' Recommended actions list ' Download report buttons (Full Report, Employer Redacted Report) ' Share score button

- Key UI components: Score display, bar chart, action list, download buttons, share button
- Primary user action: Download the PDF report or share the score
- States: Loading (generating report, loading spinner), Success (report generated, download links active), Error (report generation failed, retry button)

5. **Dashboard Screen (Authenticated Users)**

- Purpose: Show user's scan history and quick access to past reports
- Wireframe structure (top to bottom): Header with logo, navigation, user profile dropdown ' Welcome message with user's name ' Stats cards (total audits, average score, total flagged items removed) ' List of past audits (date, score, number of flagged items, view report button) ' New audit button (floating action button on mobile, top right on desktop)
- Key UI components: Stats cards, audit list, new audit button, profile dropdown, floating action button
- Primary user action: View past audits or start a new audit
- States: Loading (loading audit history), Empty (no past audits, "Start your first audit" message with CTA button), Error (failed to load history, retry button)

Technical Requirements

Stack Summary

- Platform**: Responsive website (desktop, tablet, mobile)
- Frontend**: Next.js (React 18+, App Router)
- Backend**: Firebase (Firebase Auth, Firebase Cloud Functions, Firebase Storage)
- Database**: Firebase Firestore (NoSQL)
- Theme Support**: System (auto-detects OS/browser preference)
- Visual Style**: Playful (Spotify Dark color palette, Lexend font)

Recommended Library Table

Library	Ecosystem	Purpose	Why It Fits
Firebase JS SDK v10+	NPM	Integrates Next.js with Firebase services (Auth, Firestore, Storage, Cloud Functions)	Official Google-maintained library, fully compatible with Next.js and Firebase stack
React Hook Form	NPM	Form state management and validation for upload and account forms	Lightweight, works seamlessly with Next.js, reduces boilerplate for form handling and validation
Tailwind CSS	NPM	Utility-first CSS for consistent styling and responsive design	Native Next.js support, enforces consistent spacing and design tokens, easy to implement the specified color palette and font
Framer Motion	NPM	Animations and transitions for UI elements (progress bars, screen transitions, hover states)	React/Next.js compatible, lightweight, supports the playful visual style with smooth micro-interactions
TanStack Query (React Query)	NPM	Server state management for API calls to Firebase Cloud Functions	Handles caching, loading states, and error handling for async operations, integrates natively with Next.js

PDFMake	NPM	Client-side PDF generation for audit reports	Browser-native, no server-side dependencies required for report generation, fully customizable for the required report format
React Dropzone	NPM	Drag-and-drop file upload functionality	Accessible, lightweight, works with Next.js, supports the required file validation rules

Layered Architecture Overview

- 1. **Presentation/UI Layer**: Next.js pages and React components, handles user interactions, renders UI, and manages client-side state
- 2. **State Management Layer**: TanStack Query for server state, React Hook Form for form state, React Context for authentication state
- 3. **Application/Business Logic Layer**: Next.js API routes (for server-side operations) and Firebase Cloud Functions (for AI scanning, file processing), handles business rules, validation, and workflow orchestration
- 4. **API/Service Layer**: Firebase SDK wrappers that abstract Firebase service calls (Auth, Firestore, Storage) for use in the application layer
- 5. **Data Access Layer**: Firestore security rules, data validation, and query logic, ensures secure access to stored user-specific data
- 6. **Database/Storage Layer**: Firebase Firestore (structured data for users, audits, flagged items, reports), Firebase Storage (encrypted uploaded social media archives)
- 7. **Authentication/Authorization Layer**: Firebase Auth, handles user authentication, session management, and access control for user-specific data
- 8. **External Integrations Layer**: AI scanning service (third-party API or custom model deployed on Firebase ML) for risk categorization of social media content

Folder Structure

```
clearrecord-ai/  
  public/                # Static assets  
    fonts/               # Lexend font files  
    images/              # UI icons, platform logos  
    favicon.ico  
  src/  
    app/                 # Next.js App Router pages  
      (auth)/            # Authenticated routes  
        dashboard/      # Dashboard screen
```

audit/	# Audit history and details
report/	# Report generation and download
(public)/	# Public routes
upload/	# Upload screen
scan/	# Scan progress screen
review/	# Risk review feed screen
score/	# Score and report screen
login/	# Login/signup screen
layout.tsx	# Root layout
page.tsx	# Root page (redirects to upload or dashboard)
components/	# Reusable React components
ui/	# Base UI components (buttons, inputs, cards, etc.)
upload/	# Upload-related components
scan/	# Scan progress components
review/	# Review feed components
score/	# Score display components
layout/	# Header, footer, navigation components
lib/	# Utility and service files
firebase/	# Firebase SDK initialization and service wrappers
auth.ts	
firestore.ts	
storage.ts	
functions.ts	
utils/	# Helper functions (score calculation, file validation, etc.)
hooks/	# Custom React hooks
constants/	# Static constants (risk categories, color tokens, etc.)
styles/	# Global styles and Tailwind config
globals.css	
tailwind.config.ts	
types/	# TypeScript type definitions
user.ts	
audit.ts	
flaggedItem.ts	
report.ts	
.env.local	# Local environment variables
.env.production	# Production environment variables
firebase.json	# Firebase project configuration
firestore.rules	# Firestore security rules
next.config.js	# Next.js configuration
package.json	
tsconfig.json	
tailwind.config.ts	

Key Data Flow

(End-to-end primary workflow: Upload ’ Scan ’ Review ’ Generate Report)

User ’ (1) Selects social media archive file ’ (2) Next.js frontend validates file type/size ’ (3) Frontend uploads file to Firebase Storage ’ (4) Firestore creates new Audit record with status 'pending' ’ (5) Frontend triggers Firebase Cloud Function to start scan ’ (6) Cloud Function parses archive, extracts content ’ (7) Cloud Function sends content to AI scanning service ’ (8) AI service returns risk categories and levels for each content item ’ (9) Cloud Function saves flagged items to Firestore, updates Audit status to 'completed' and calculates Digital Hygiene Score ’ (10) Frontend polls Firestore for scan status update ’ (11) Frontend renders Risk Review Feed with flagged items ’ (12) User takes action on items (mark reviewed, delete, untag) ’ (13) Frontend updates FlaggedItem records in Firestore ’ (14) Frontend recalculates Digital Hygiene Score in real-time ’ (15) User requests PDF report ’ (16) Frontend generates PDF using PDFMake with audit data ’ (17) Frontend uploads PDF to Firebase Storage, saves Report record to Firestore ’ (18) User downloads PDF report

Firestore Data Model (NoSQL, compatible with Firebase)

Collection: `users`

Field Name	Data Type	Required	Constraints	Purpose
user_id	string	Yes	Primary key, auto-generated UUID	Unique user identifier
email	string	Yes	Unique, valid email format	User login email
full_name	string	Yes	Max 255 characters	User display name
auth_provider	string	Yes	Enum: `email`, `google`	Authentication provider used
password_hash	string	No	Required if auth_provider is `email`	Hashed user password
created_at	timestamp	Yes	Default: current timestamp	Account creation time
updated_at	timestamp	Yes	Default: current timestamp, auto-updated on edit	Last account update time

Collection: `audits`

Field Name	Data Type	Required	Constraints	Purpose

audit_id	string	Yes	Primary key, auto-generated UUID	Unique audit identifier
user_id	string	Yes	Foreign key to `users.user_id`	Owner of the audit
upload_date	timestamp	Yes	Default: current timestamp	Date archive was uploaded
scan_status	string	Yes	Enum: `pending`, `processing`, `completed`, `failed`, Default: `pending`	Current scan status
digital_hygiene_score	number	No	Range: 0-100, only set when scan_status is `completed`	Final Digital Hygiene Score
total_items_scanned	number	No	Only set when scan completes	Total number of content items scanned
total_flagged_items	number	No	Only set when scan completes	Total number of flagged items found
archive_file_path	string	Yes	Max 1024 characters, path to file in Firebase Storage	Path to uploaded archive file
created_at	timestamp	Yes	Default: current timestamp	Audit creation time

Collection: `flagged\_items`

Field Name	Data Type	Required	Constraints	Purpose
flagged_item_id	string	Yes	Primary key, auto-generated UUID	Unique flagged item identifier
audit_id	string	Yes	Foreign key to `audits.audit_id`	Audit this item belongs to
content_text	string	No	Max 5000 characters	Text content of the flagged post/comment

media_url	string	No	Max 1024 characters, URL to media in Fire- base Storage	URL to associated me- dia (image/video)
original_platform	string	Yes	Enum: `facebook`, `in- stagram`, `twitter`, `linkedin`, `tiktok`	Platform the content was posted on
post_date	timestamp	Yes	Date the original content was posted	Original post date
risk_category	string	Yes	Enum: `workplace_hostility`, `aggressive_debates`, `profanity`, `substance_refer- ences`, `discriminatory_lan- guage`, `unprofessional_behav- ior`, `confidential_info_shar- ing`, `fake_news`, `extremist_content`, `inappropriate_media`, `privacy_violations`, `dishonest_behavior`, `violent_threats`	Risk category assigned to the content
risk_level	string	Yes	Enum: `high`, `medium`, `safe`	Severity level of the risk
user_action	string	Yes	Enum: `pending`, `re- viewed`, `deleted`, `un- tagged`, Default: `pend- ing`	Action taken by the user on the item
created_at	timestamp	Yes	Default: current time- stamp	Item creation time

Collection: `reports`

Field Name	Data Type	Required	Constraints	Purpose

report_id	string	Yes	Primary key, auto-generated UUID	Unique report identifier
audit_id	string	Yes	Foreign key to `audits.audit_id`	Audit the report is for
report_type	string	Yes	Enum: `full`, `employer_redacted`	Type of report generated
pdf_url	string	Yes	Max 1024 characters, URL to PDF in Firebase Storage	URL to the generated PDF report
generated_at	timestamp	Yes	Default: current timestamp	Date report was generated
expires_at	timestamp	Yes	Default: 30 days after generation	Date report expires and is deleted

Indexes:

- `audits` collection: Index on `user\_id` (fetch all audits for a user), index on `scan\_status` (filter pending scans)
- `flagged\_items` collection: Index on `audit\_id` (fetch all items for an audit), index on `risk\_level` (sort by risk level), index on `risk\_category` (filter by category)

Build & Release Checklist

1. **Local Development Setup**

- Clone repository
- Install Node.js 18+ and npm/yarn
- Install Firebase CLI and log in to Firebase account
- Create Firebase project and configure `firebase.json` with project ID
- Copy `.env.example` to `.env.local` and fill in Firebase config values (API key, project ID, etc.)
- Run `npm install` to install dependencies
- Run `firebase emulators:start` to start local Firebase emulators (Firestore, Storage, Auth, Functions)
- Run `npm run dev` to start Next.js development server
- Verify local development server runs at `http://localhost:3000`

2. **Environment Variables & Secrets**

- Ensure all Firebase config values are stored in environment variables, not hardcoded
- Store AI scanning service API key (if using third-party) in Firebase environment variables for Cloud Functions
- Ensure production environment variables are set in Firebase hosting and Cloud Functions config

3. **Dependency Installation**

- Run ``npm install`` for all dependencies
- Run ``npm audit fix`` to resolve known security vulnerabilities
- Verify all dependencies are compatible with Next.js 14+ and Firebase SDK v10+

4. **Database Setup & Migrations**

- Deploy Firestore security rules using ``firebase deploy --only firestore:rules``
- Deploy Firestore indexes using ``firebase deploy --only firestore:indexes``
- Initialize Firestore collections with required fields (no seed data required for initial launch)

5. **Development Server Execution**

- Run ``npm run dev`` for local development
- Verify all core screens load correctly, Firebase services connect successfully
- Test file upload, scan, review, and report generation workflows locally using emulators

6. **Linting & Formatting**

- Run ``npm run lint`` to check for code style issues
- Run ``npm run format`` to format code with Prettier
- Ensure no linting errors before committing code

7. **Testing**

- Run unit tests with ``npm run test:unit`` (target 80% code coverage for core business logic)
- Run integration tests with ``npm run test:integration`` (test Firebase service integrations, scan workflow)
- Run end-to-end tests with ``npm run test:e2e`` (test full user workflows: upload ' scan ' review ' report download)

8. **Production Build**

- Run ``npm run build`` to generate optimized Next.js production build
- Verify build completes without errors

- Test production build locally with ``npm run start``

9. ****Deployment****

- Deploy Next.js app to Firebase Hosting using ``firebase deploy --only hosting``
- Deploy Firebase Cloud Functions using ``firebase deploy --only functions``
- Deploy Firestore rules and indexes using ``firebase deploy --only firestore``
- Verify production site is accessible at custom domain
- Test all core workflows in production environment

10. ****Database Migration in Production****

- No schema migrations required for initial launch (Firestore is schemaless)
- For future feature updates, deploy new Firestore rules and indexes via Firebase CLI
- Back up Firestore data regularly using Firebase scheduled backups

11. ****Environment Configuration****

- Verify production environment variables are set correctly in Firebase console
- Enable Firebase Authentication (email/password and Google providers)
- Configure Firebase Storage security rules to restrict access to user-specific archives
- Set up Firebase Cloud Functions region for low latency

12. ****Monitoring & Logging****

- Enable Firebase Crashlytics for frontend error tracking
- Enable Firebase Cloud Functions logging for backend error tracking
- Set up Firebase Performance Monitoring to track page load times and API latency
- Configure alerts for scan failure rates, error rates, and service downtime

13. ****Security Checks****

- Run ``npm audit`` to check for dependency vulnerabilities
- Verify Firestore security rules restrict access to user-specific data only
- Verify Firebase Storage rules prevent unauthorized access to uploaded archives
- Ensure all API endpoints are authenticated and authorized

- Verify PDF reports do not expose sensitive user data in employer redacted mode

14. ****Rollback Considerations****

- Use Firebase Hosting rollback feature to revert to previous deployment if issues are found
- Keep previous Cloud Functions versions deployed for quick rollback
- Maintain database backups for 30 days to restore data if needed
- Document rollback steps in internal runbook